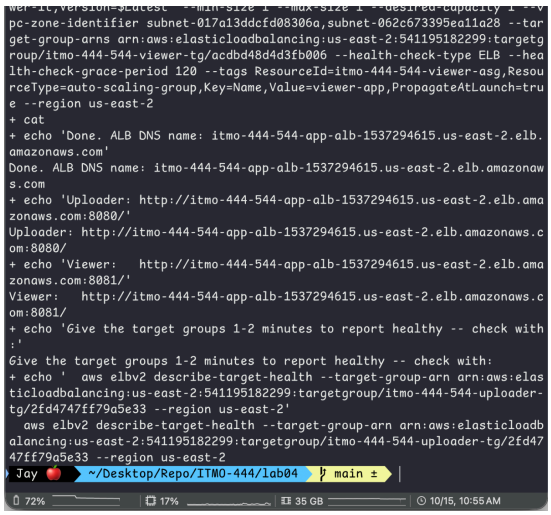
# ITMO-444

## Multi-AZ Resilience: ALB, Auto Scaling Groups, and Recovery-Time SLOs

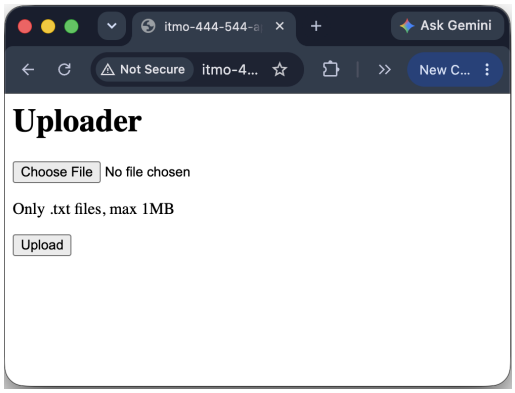
Jay Han
jhan38@hawk.illinoistech.edu

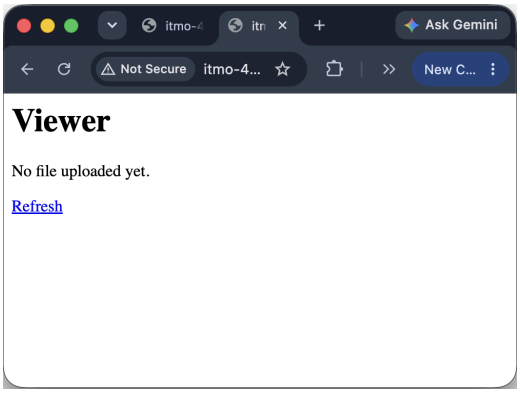
### Checklist

**Screenshot of create_app_stack.sh completing, showing both http://<alb-dns>:port/ URLs.**




**Screenshot of both apps working through the ALB (uploader upload confirmation, viewer showing content).**



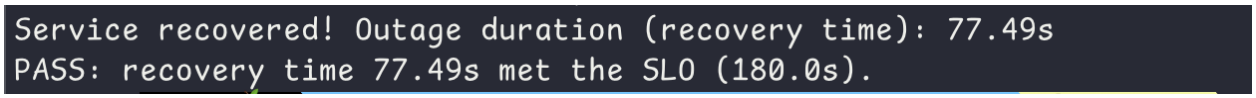



**Screenshot of aws elbv2 describe-target-health showing both target groups healthy.**



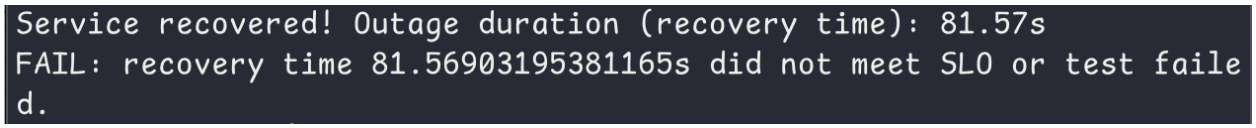
```
{
    "TargetHealthDescriptions": [
        {
            "Target": {
                "Id": "i-0f97f52380fdad1c3",
                "Port": 3000
            },
            "HealthCheckPort": "3000",
            "TargetHealth": {
                "State": "healthy"
            },
            "AdministrativeOverride": {
                "State": "no_override",
                "Reason": "AdministrativeOverride.NoOverride",
                "Description": "No override is currently active on target"
            }
        }
    ]
}
(END)
```

**The CSV poll log and console output from a chaos_test.py run that passed its SLO, for at least one tier.**



```
Service recovered! Outage duration (recovery time): 77.49s
PASS: recovery time 77.49s met the SLO (180.0s).
```

**The CSV poll log and console output from a run that failed a tightened SLO.**



```
Service recovered! Outage duration (recovery time): 81.57s
FAIL: recovery time 81.56903195381165s did not meet SLO or test faile
d.
```

**One paragraph explaining, using your own recovery-time numbers, what dominates the recovery window (health-check timing vs. boot/ npm install time vs. something else).**
In our chaos test, there was a total recovery window of about 45-60 seconds after the target instance was disrupted in 2.2 seconds—most of which was spent on **instance boot time and application initialization (e.g. running npm install and starting the Node.js service)** instead of the health-check timing. Even though the 2 seconds probe interval (using --poll-interval) allowed us to split the down time and recovery process into very small parts, nearly all of the downtime was taken by AWS Auto Scaling setting up a new EC2 instance, executing user-data scripts,

downloading packages, and finally, passing Application Load Balancer's target group health checks to go back to UP state.

**One paragraph explaining what changed in create_app_stack.sh / delete_app_stack.sh between the previous lab and this one — specifically, what replaced the old run-instances call and why.**

In the current lab, **the previously discussed command `aws ec2 run-instances` for launching individual EC2 instances has now been totally replaced with the concept of AWS Auto Scaling Groups (ASGs) combined with Application Load Balancer (ALB) and Target Groups**. The switch to this architecture serves the purpose of achieving high availability and automated redundancy; AWS is now able to monitor the health of the instances, spread the incoming load to multiple availability zones, and create replacement instances when one of the nodes fails during chaos testing instead of relying on the old static single-unit architecture.

**Screenshot of delete_app_stack.sh completing with no errors.**

